



Univerzitet u Novom Sadu
Fakultet tehničkih nauka



Specifikacija projektnog zadatka

Student: Vladimir Čajka, SV 55/2024

Predmet: Paralelno programiranje

Tema projektnog zadatka: Paralelizacija genetskog algoritma za rešavanje problema putujućeg trgovca

Sadržaj

Sadržaj	2
Opis problema	3
Uvod.....	4
Cilj projekta	5
Planirana paralelizacija	5
Eksperimentalna evaluacija.....	8
Planirani eksperimenti	8
Očekivani rezultati	9
Zaključak	9

Opis problema

Problem putujućeg trgovca (Travelling Salesman Problem – TSP) predstavlja jedan od klasičnih problema kombinatorne optimizacije. Dat je skup gradova (čvorova) i međusobne udaljenosti između njih. Potrebno je pronaći najkraću moguću zatvorenu turu koja:

1. posećuje svaki grad tačno jednom,
2. vraća se u početni grad,
3. minimizuje ukupnu dužinu ture.

Udaljenosti između gradova računaju se Euklidskom formulom:

$$d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

Cilj algoritma je da pronađe redosled obilaska gradova (permutaciju indeksa/ID-jeva) koji minimizuje ukupnu dužinu zatvorene ture. Izlaz rešenja je:

- najbolja pronađena tura (redosled ID-jeva gradova),
- ukupna dužina te ture,
- pomoćni fajlovi (istorija konvergencije i/ili grafički prikaz).

Pošto je TSP NP-težak problem, za veće instance je nepraktično koristiti egzaktne metode. Zbog toga se koriste heuristike i metaheuristike. U ovom projektu koristiće se genetski algoritam kao evolutivna metaheuristika. U projektu će se koristiti *Berlin52* dataset za problem putujućeg trgovca koji sadrži koordinate 52 grada.

Uvod

Genetski algoritam (GA) je optimizaciona metoda inspirisana principima biološke evolucije: selekcijom, ukrštanjem i mutacijom. GA održava populaciju kandidata (jedinki) i kroz više generacija postepeno poboljšava rešenja. Osnovna ideja je da se dobri delovi rešenja prenose i kombinuju kroz ukrštanje, dok mutacija održava raznolikost populacije i omogućava izlazak iz lokalnih optimuma.

U kontekstu TSP-a, osnovni izazov je reprezentacija rešenja i definisanje operatora tako da generisana deca budu validne ture (tj. permutacije bez ponavljanja gradova). U projektu koristiće se reprezentacija:

- **hromozom** = permutacija svih gradova (npr. [0, 5, 2, ...]),
- **gen** = jedan grad (indeks grada),
- **fitness/ciljna funkcija** = ukupna dužina zatvorene ture (manje je bolje).

Tok GA u svakoj generaciji može se opisati sledećim koracima:

1. Izračunavanje vrednosti ciljne funkcije za svaku jedinku (dužina ture).
2. **Elitizam**: najbolji deo populacije se direktno prenosi u sledeću generaciju.
3. **Selekcija roditelja**: biraju se jedinke koje će učestvovati u ukrštanju.
4. **Ukrštanje (crossover)**: od dva roditelja prave se jedna/dve nove jedinke.
5. **Mutacija**: nasumična izmena potomaka sa zadatom verovatnoćom.
6. Formiranje nove populacije i ponavljanje procesa.
7. Zaustavljanje nakon maksimalnog broja generacija ili nakon duže stagnacije (early stop).

Prednost GA pristupa je fleksibilnost: izbor selekcije, ukrštanja, mutacije i parametara može se prilagoditi tipu problema. Kod TSP-a se pokazalo da operatori koji čuvaju strukturu ture i susedstva gradova daju značajno bolje rezultate.

Cilj projekta

Cilj projekta je implementacija serijske i paralelne verzije genetskog algoritma za rešavanje problema putujućeg trgovca - TSP.

Serijska verzija će biti implementirana u C++ jeziku, dok će paralelna verzija koristiti Intel TBB biblioteku. Nakon implementacije biće izvršeno poređenje performansi obe verzije kroz merenje vremena izvršavanja, ubrzanja i efikasnosti paralelizacije.

Poseban fokus projekta biće na identifikaciji delova genetskog algoritma koji se mogu nezavisno izvršavati i njihovoj paralelizaciji pomoću odgovarajućih TBB mehanizama.

Planirana paralelizacija

Paralelizacija će biti urađena pomoću Intel TBB biblioteke. Planirano je da se paralelizuju delovi algoritma u kojima se više operacija može izvršavati nezavisno.

Paralelna izgradnja matrice udaljenosti

Na početku programa formira se matrica udaljenosti između svih parova gradova.

Pošto se udaljenost za svaki par gradova može izračunati nezavisno, ovaj deo može da se paralelizuje pomoću:

tbb::parallel_for

Ova faza se izvršava samo jednom, pa se ne očekuje da bude glavni izvor ubrzanja, ali je prirodan primer data-parallel obrade.

Paralelno generisanje početne populacije

Svaka jedinka početne populacije predstavlja nasumičnu validnu permutaciju gradova. Pošto se jedinke mogu generisati nezavisno, ovaj deo se može paralelizovati po indeksima populacije.

Planirana TBB konstrukcija:

tbb::parallel_for

Pri implementaciji će se voditi računa da svaka nit koristi lokalni generator slučajnih brojeva, kako ne bi došlo do konkurentnog pristupa jednom zajedničkom generatoru.

Paralelna evaluacija populacije

Evaluacija jedinke podrazumeva računanje ukupne dužine njene ture. Pošto se dužina svake ture računa nezavisno od ostalih jedinki, evaluacija populacije je jedan od najprirodnijih delova za paralelizaciju.

Planirana TBB konstrukcija:

tbb::parallel_for

Ovaj deo će predstavljati osnovnu paralelizaciju u projektu.

Paralelno generisanje nove populacije

Nakon prenosa elitnih jedinki, ostatak nove populacije nastaje selekcijom roditelja, ukrštanjem i mutacijom.

Svako dete može da se generiše nezavisno od ostale dece, pa se ovaj deo može paralelizovati po indeksima nove populacije.

Planirana TBB konstrukcija:

tbb::parallel_for

Ovo je jedan od najvažnijih delova projekta, jer se izvršava u svakoj generaciji i obuhvata računski značajne operacije: izbor roditelja, crossover i mutaciju.

Paralelno sortiranje populacije

U svakoj generaciji populacija se sortira po dužini ture kako bi se izdvojile najbolje jedinke i primenio elitizam.

Za ovaj deo planirano je korišćenje:

tbb::parallel_sort

Ova paralelizacija ima najviše smisla za veće populacije, gde sortiranje može postati značajan deo ukupnog vremena izvršavanja.

Paralelno računanje statistike

Radi analize rada algoritma računaće se osnovne statistike po generaciji:

- najbolja dužina ture
- prosečna dužina ture
- eventualno najgora dužina ture

Za ove agregacije planirano je korišćenje:

tbb::parallel_reduce

Na taj način se koristi redukcionni oblik paralelizma, gde svaka nit računa lokalni rezultat, a zatim se lokalni rezultati spajaju u globalni.

Island model kao napredna paralelizacija

Kao napredna verzija projekta planira se implementacija island modela.

Umesto jedne populacije, koristiće se više nezavisnih populacija, odnosno ostrva. Svako ostrvo evoluira nezavisno određeni broj generacija, a zatim se periodično vrši migracija najboljih jedinki između ostrva.

Primer migracije:

Island 1 -> Island 2

Island 2 -> Island 3

Island 3 -> Island 4

Island 4 -> Island 1

Planirana TBB konstrukcija:

tbb::parallel_for

Ovaj model omogućava coarse-grained paralelizam, jer svaka populacija može da se razvija nezavisno. Migracija predstavlja tačku sinhronizacije između ostrva.

Island model je posebno koristan jer se koristi dataset od 52 grada. Kod manje TSP instance pojedinačna evaluacija ture nije previše skupa, pa veće populacije, veći broj generacija i više nezavisnih populacija daju dovoljno posla da se paralelizacija bolje iskoristi.

Eksperimentalna evaluacija

Biće implementirane i upoređene dve osnovne verzije:

1. serijska C++ verzija
2. paralelna C++ verzija pomoću TBB biblioteke

Glavno poređenje biće između serijske i paralelne C++ verzije.

Merice se:

- ukupno vreme izvršavanja
- najbolja pronađena dužina ture
- prosečna dužina ture po generacijama
- speedup
- efikasnost

Speedup se računa kao:

$$S = T_{\text{serijski}} / T_{\text{paralelni}}$$

Efikasnost se računa kao:

$$E = S / p$$

gde je p broj niti.

Planirani eksperimenti

Pošto dataset sadrži 52 grada, za ozbiljnije merenje paralelizacije koristiće se veće populacije i veći broj generacija.

Planirani eksperimenti uključuju:

Eksperiment 1 — uticaj broja niti

Fiksiraju se veličina populacije i broj generacija, a menja se broj niti:

$$threads = 1, 2, 4, 8, 12, 16$$

Cilj je da se izmeri ubrzanje paralelne verzije.

Eksperiment 2 — uticaj veličine populacije

Fiksiraju se broj niti i broj generacija, a menja se veličina populacije:

$$population_size = 1\ 000, 5\ 000, 10\ 000, 20\ 000$$

Cilj je da se pokaže da se paralelizacija više isplati kada je količina posla veća.

Eksperiment 3 — island model

Poredi se jedna velika populacija sa više manjih nezavisnih populacija:

1 island x 16 000 jedinki

2 islands x 8 000 jedinki

4 islands x 4 000 jedinki

8 islands x 2 000 jedinki

Cilj je da se ispita uticaj island modela na vreme izvršavanja i kvalitet pronađenog rešenja.

Očekivani rezultati

Očekuje se da paralelizacija neće dati veliko ubrzanje za male populacije, jer je problem sa 52 grada relativno mali i overhead paralelnih zadataka može biti značajan.

Veće ubrzanje se očekuje kod većih populacija i većeg broja generacija, posebno u fazi generisanja nove populacije i kod island modela.

Takođe se očekuje da island model može doprineti boljem očuvanju raznolikosti populacije i smanjenju rizika od prerane konvergencije.

Zaključak

Predloženi projekat bavi se paralelizacijom genetskog algoritma za rešavanje problema putujućeg trgovca. Fokus projekta nije samo na implementaciji samog algoritma, već na analizi delova algoritma koji se mogu izvršavati paralelno.

Planirana paralelizacija obuhvata evaluaciju populacije, generisanje početne i nove populacije, sortiranje, računanje statistike i opciono island model. Rezultati će se analizirati kroz poređenje serijske i paralelne C++ verzije, merenje vremena izvršavanja, ubrzanja, efikasnosti i kvaliteta pronađenog rešenja.